COMP2511

Tutorial 5

Last Week's Tutorial

- Design Principles
- Introduction to Design Patterns

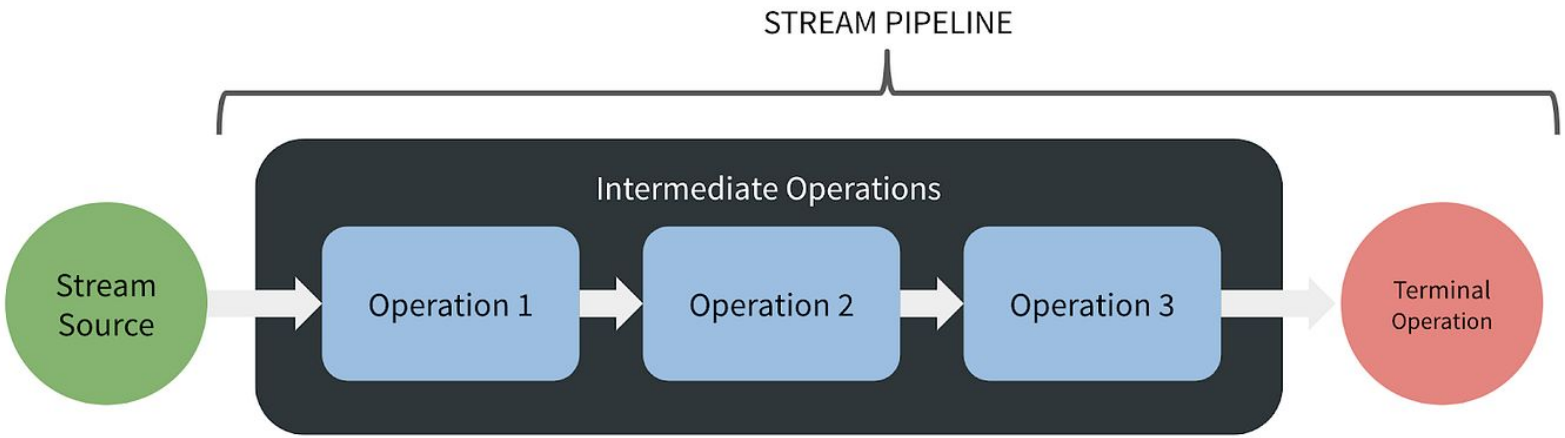
This Week's Tutorial

- Streams
- More Design Patterns
  - Strategy Pattern
  - Observer Pattern

Streams

- A stream is an abstracted sequence of elements. Collections in Java (eg. `List`, `Map`, `Set`) can be transformed into streams using `.stream()`.
- Elements in a stream can only be interacted with using particular methods - these work specifically on streams, and provide abstractions.
- Two types of stream operations:
  - **Intermediate** operations: transforms the stream into another stream. These operations can be chained together.
    - `map`, `filter`, `distinct`
  - **Terminal** operations: marks the end of a stream pipeline and produces a final result.
    - `reduce`, `sum`, `toList`

Streams


STREAM PIPELINE
Intermediate Operations
Stream Source
Operation 1
Operation 2
Operation 3
Terminal Operation

Stream Operations

- `filter`: only keeps elements that satisfy a specific condition
  - `.filter(x -> x < 5)`
- `map`: applies a function to all elements in the stream, modifying its value
  - `.map(x -> x * 3)`
- `reduce`: combines elements into a single result
  - `.reduce(1, (a, b) -> a * b)`
- `distinct`: removes duplicate elements
- `toList`: convert stream into a list (returned list is **immutable**)

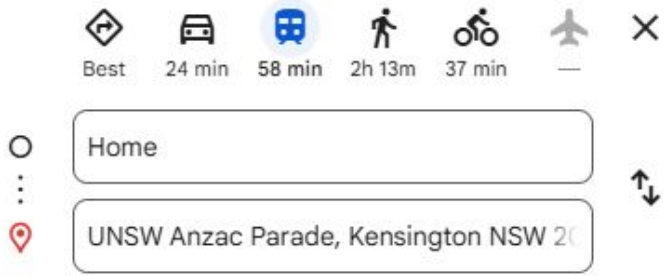
```
List<Integer> l = List.of(1, 2, 3, 4, 5, 6);
l = l.stream().filter(x -> x < 5).map(x -> x * 3).toList();
```

```
[3, 6, 9, 12]
```

Live Example: Streams

- Complete the tasks in **src/stream** using streams.

Strategy Pattern

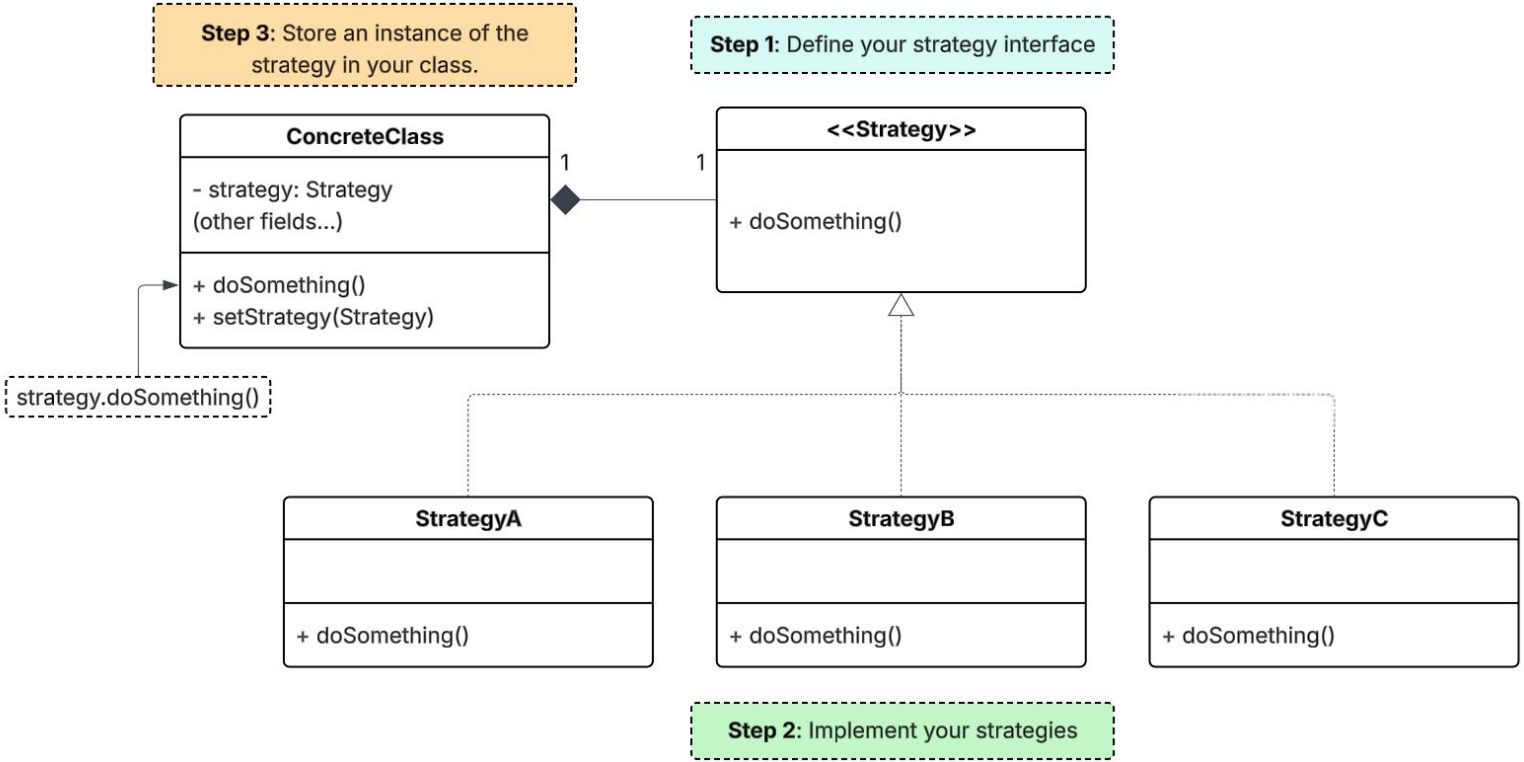


- Consider how you can alternate between different modes of transport while planning a route on Google Maps.
  - The specific details of the route, e.g. the distance you have to cover, the turns you have to take, any costs incurred etc. may be **very** different across the modes of transport.
  - Regardless, the common aspect of each of these routes is that they all go the same destination. You can think of them as being “different ways to do the same thing”, and being able to easily switch between them is what the Strategy Pattern is all about.

Strategy Pattern

- The Strategy Pattern is a (behavioural) design pattern where multiple implementations are defined for a specific action, and the class is able to switch between these implementations at **run-time**.
- Rather than putting all of the different implementations in a single class, each implementation is separated into different classes called **strategies**.
  - Each strategy should implement a specific **interface** (or inherit an abstract class) that defines the methods common to each strategy.
  - The original class stores an **instance** of a particular strategy, and executes instructions by calling the appropriate method(s) on the instance.
  - Switching between strategies is done by changing this field, e.g. through a setter.
  - Importantly, the functionality of the object differs according to which strategy is being used, due to polymorphism.

Strategy Pattern: Implementation



Live Example: Strategy Pattern

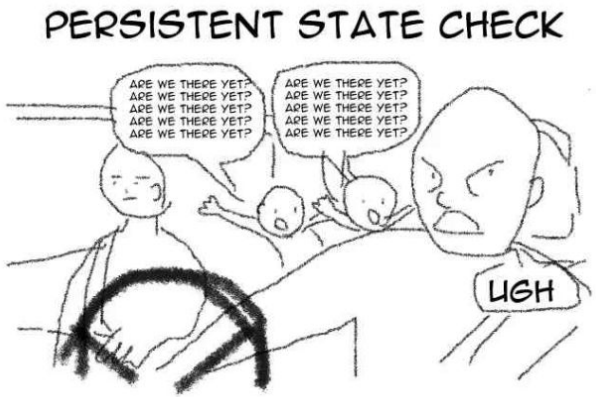
- Inside **src/restaurant** is a restaurant payment system with the following requirements:
  - The restaurant has a menu (a list of meals). Each meal has a name and price.
  - Users can enter their order as a series of meals, and the system returns their cost.
- The prices on meals vary in different circumstances. The restaurant has three different price settings (so far):
  - **Standard** - normal rates
  - **Holiday** - 15% surcharge on all items for all customers
  - **Happy Hour** - registered members get a 40% discount, standard customers get 30%
- Refactor the code using the Strategy Pattern to handle the three settings, then implement a new **Prize Draw** Strategy - every 10th customer (since the start of the promotion) gets their meal for free!

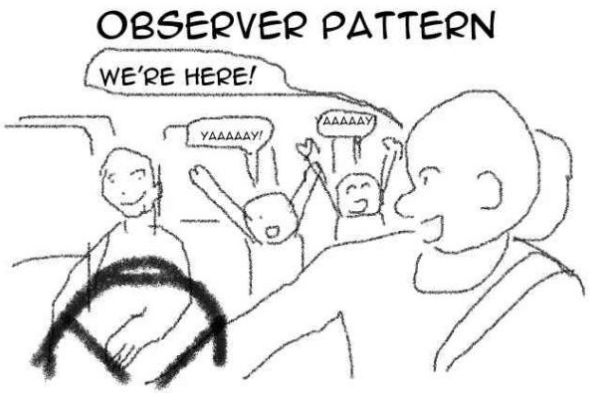
Observer Pattern

- If you were subscribed to a YouTube channel and wanted to watch that channel's videos as soon as they came out, would you rather:
  - Keep refreshing the YouTube channel every 5 seconds?
  - Receive a notification whenever a new video has been uploaded?
- The latter case is more sensible - when a channel publishes a new video, its subscribers get notified.
- We can frame this scenario in a particular way: There is an important **event** (a new video being uploaded) that some people are waiting on. As soon as that event happens, everyone waiting will get **notified** and respond to it; this is the idea of the Observer Pattern.

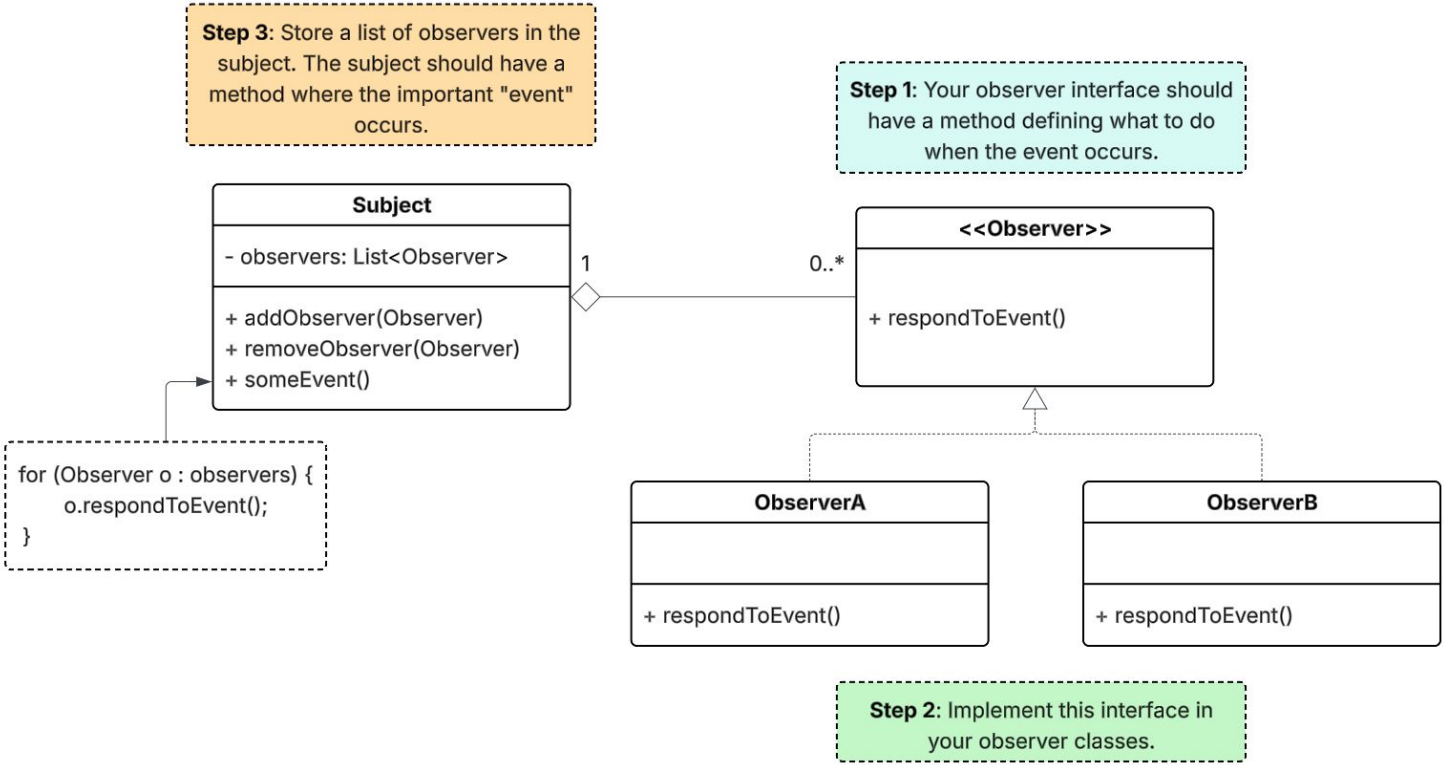
Observer Pattern

- The Observer Pattern is a (behavioural) design pattern where a subject maintains a list of subscribers, who are **notified** once an important event happens with the subject.





Observer Pattern: Implementation



Live Example: Observer Pattern

- In **src/youtube**, create a model for the following requirements of a YouTube-like video creating and watching service using the Observer Pattern.
  - A YouTuber has a name, subscribers and videos.
  - A User has a name and can subscribe to any YouTuber.
  - When a YouTuber posts a new video, the video gets added to the 'Watch Later' lists of all Users subscribed to that YouTuber.
  - Write a simple test with print statements inside YouTube.java.